lecture_30

October 23, 2026

1 Lecture 30: Ant Colony Optimization - Motivation & Pseudocode

---

```
{note} Simulated Annealing (Lectures 24-26) refines one solution; the
Genetic Algorithm (Lectures 27-29) evolves a population through selection
and recombination. **Ant Colony Optimization (ACO)**, Lecture 23's third
paradigm, takes a different approach again: a colony of simple agents ("ants")
repeatedly *construct* candidate solutions from scratch, one component at a time,
coordinating with each other only indirectly, through a shared memory called
pheromone. This lecture develops ACO as generic pseudocode for any problem whose
solutions can be assembled from discrete components; Lecture 31 adapts it to the
Ackley function, and Lecture 32 instantiates it for routing.
```

Learning Objectives

By the end of this notebook, you will be able to: 1. Explain how indirect coordination through pheromone (stigmergy) lets a colony of simple agents solve a combinatorial problem with no central controller. 2. Read and interpret Ant Colony Optimization's pseudocode, including the probabilistic construction rule and the evaporation/deposit update. 3. Hand-trace one iteration of Ant Colony Optimization — each ant's construction and the resulting pheromone update — given a small problem and a sequence of random draws.

Prerequisites: Genetic Algorithm - Benchmarking (Lecture 29); Metaheuristics (Lecture 23).

Estimated time: 50 minutes

1.1 Why Ant Colony Optimization?

A real ant colony finds short paths between its nest and a food source with no map, no leader, and no ant that can see the whole trail network. Each ant deposits a chemical **pheromone** as it walks, and is more likely to follow a path already carrying more pheromone. Shorter paths get walked — and so reinforced — more often per unit time than longer ones, so pheromone accumulates faster on them; over many ants and many trips, the colony converges on something close to the shortest route, purely through this indirect, trail-mediated feedback. This mechanism — agents coordinating not by talking to each other, but by modifying and reacting to a shared environment — is called **stigmergy**.

Ant Colony Optimization turns this into a general search algorithm. The key observation is that a solution to many combinatorial problems can be **assembled one decision at a time** from a finite set of **solution components**: the next stop on a route, the next job in a schedule, the value assigned to the next decision variable, the next item added to a subset. Each iteration, a colony of m artificial ants each **constructs** a complete candidate solution, choosing every next component probabilistically — biased toward components carrying more pheromone (what has worked for the colony before) and toward components that simply look good on their own (a problem-specific **heuristic** rule of thumb, such as "nearer stops are better"). Once every ant has finished, pheromone **evaporates** a little everywhere, and every ant **deposits** fresh pheromone on the components of the solution it just built, in an amount that grows with that solution's quality. Repeated over many iterations, this reinforce-what-worked, fade-what-didn't cycle concentrates the colony's construction choices onto increasingly good solutions.

`{note} Compare this to Simulated Annealing and the Genetic Algorithm: SA refines one solution by moving through a neighbourhood; GA evolves a population by recombining existing solutions. ACO does neither - it **builds new solutions from scratch every iteration**, one component at a time, guided by a shared memory (pheromone) that persists and evolves across iterations. Any problem whose solutions can be built as a sequence of discrete choices fits this mould directly - routing is the textbook example, but by no means the only one.`

1.2 Notation

Symbol	Meaning
C	The set of **solution components** a solution can be assembled from
s^p	A partial solution under construction (a set or sequence of components chosen so far)
$N(s^p)$	The **feasible components** — those that can extend s^p without violating a constraint
s_k	The complete solution constructed by ant k in the current iteration
s^*	Best solution found so far
$f(s)$	Objective function value (to be minimized, and assumed positive) at solution s
τ_c, τ_o	Pheromone level on component c; its initial value
η_c	Heuristic desirability of component c — a problem-specific estimate, higher for components that look better in isolation
α, β	Pheromone and heuristic importance weights
ρ	Evaporation rate ($0 < \rho < 1$)
Q	Pheromone deposit constant
m	Number of ants
$\Lambda \sim U(0,1)$	A random number drawn uniformly from $[0,1]$, used to pick the next component

Construction rule: an ant holding partial solution s^p extends it with component $c \in N(s^p)$ with probability

$$p(c \mid s^p) = \frac{\tau_c^\alpha \, \eta_c^\beta}{\sum_{c' \in N(s^p)} \tau_{c'}^\alpha \, \eta_{c'}^\beta}$$

In practice this is a **roulette wheel**: list the feasible components in a fixed order, draw λ, and pick the first component whose cumulative probability exceeds λ.

This notation deliberately leaves C, $N(s^p)$, and η_c unspecified — like the Genetic Algorithm's operators in Lecture 27, they are **pluggable**, and choosing them is how ACO is tailored to a problem. The engine below stays the same either way:

Problem	Component c	Feasible set $N(s^p)$	Heuristic η_c
Routing (TSP)	Edge (i, j): travel from the current stop i to stop j	Edges to stops not yet visited	$1/d_{ij}$ — shorter edges look better
Binary decisions	Assignment $x_j = v$ for the next variable j	$\{x_j = 0,\ x_j = 1\}$	A prior guess at which value is better
Scheduling	Job j placed in the next position	Jobs not yet scheduled	E.g. $1/\text{processing time}_j$
Assignment	Task i assigned to agent j	Agents with remaining capacity	$1/\text{cost}_{ij}$

Every row can be drawn as a **construction graph** — components as the edges (or nodes) an ant walks along, pheromone living on each one — which is why ACO is usually described as an algorithm "on a graph", even for problems that have nothing to do with roads.

`{tip} α and β play the same exploration/exploitation role as every other parameter this module has calibrated: $\alpha=0$ ignores pheromone entirely (every ant becomes a randomised greedy construction, guided by the heuristic alone and never learning from the colony); $\beta=0$ ignores the heuristic entirely (ants follow pheromone alone, however the colony happened to lay it). Setting both above zero blends "what has worked for the colony so far" with "what looks locally promising right now" - exactly the same blend SA's acceptance rule and the Genetic Algorithm's fitness-weighted selection are built from, expressed here as a single probability instead of an accept/reject test or a selection rule.`

1.3 Pseudocode

1. **Procedure** ACO$(C, N, \eta, f, \tau_o, \alpha, \beta, \rho, Q, m)$
2. $\tau_c \leftarrow \tau_o$ for every $c \in C$ // initialise pheromone uniformly
3. $f(s^*) \leftarrow \infty$ // no solution found yet
4. **while** !converged **do**

5. **for** ant $k = 1, \dots, m$ **do**
6. $s^p \leftarrow \emptyset$ // start from an empty partial solution
7. **while** s^p is not complete **do**
8. $c \overset{p}{\leftarrow} N(s^p)$ // draw λ; pick a feasible component using the construction rule
9. $s^p \leftarrow s^p \cup \{c\}$ // extend the partial solution
10. **end while**
11. $s_k \leftarrow s^p$
12. **end for**
13. $\tau_c \leftarrow (1-\rho)\,\tau_c$ for every $c \in C$ // evaporate
14. **for** ant $k = 1, \dots, m$ **do**
15. $\tau_c \leftarrow \tau_c + Q/f(s_k)$ for every $c \in s_k$ // deposit, more for better solutions
16. **end for**
17. $s \leftarrow \text{argmin}\{f(s_k) : k = 1, \dots, m\}$ // iteration best
18. **if** $f(s) < f(s^*)$ **then**
19. $s^* \leftarrow s$
20. **end if**
21. **end while**
22. **return** s^*

```
{caution} Notice there is no explicit "current solution" or "population" carried
between iterations the way $s$ or $\boldsymbol{s}$ were in Lectures 24 and 27
- the only thing that persists from one iteration to the next (apart from the
incumbent $s^*$) is the pheromone $\tau$. Every ant constructs its solution
completely fresh each iteration; it is the *pheromone*, not any individual ant,
that accumulates the colony's learning over time. Notice also that pheromone is
updated only after *all* $m$ ants have finished (lines 13-16) - ants within the
same iteration all construct using the same pheromone.
```

Line 15's deposit $Q/f(s_k)$ is the original **Ant System** rule, and needs $f > 0$. Later ACO variants change only lines 13-16 — e.g. letting only the iteration-best or best-so-far ant deposit, or clamping τ between a minimum and a maximum (the MAX-MIN Ant System) — while the construction loop stays the same.

1.4 Applying ACO to Continuous Problems

Every part of ACO's construction rule assumes a **finite set of discrete components**, each carrying its own pheromone value. None of that exists for a continuous function like the Ackley function Lecture 31 will need for its first hands-on implementation — a real-valued variable can take infinitely many values, so there is no finite component set to deposit pheromone on.

```
{note} **ACOR** (Ant Colony Optimization for continuous domains; Socha & Dorigo,
2008) resolves this by replacing the pheromone values with an **archive** of
the $k$ best real-valued solutions found so far, each carrying a weight based
on its rank (better solutions weighted more heavily). A new candidate is built
dimension-by-dimension: for each dimension, one archive member is picked at
random (favouring higher-weighted, better-ranked members), and a new value is
sampled from a Gaussian distribution centred on that member's value, with a
spread based on how close together the archive's members already are in that
```

`dimension. New candidates are merged into the archive, which is then trimmed back to its $k$ best - playing the same reinforce-what-worked, fade-what-didn't role that evaporation and deposit play for pheromone on discrete components.`

Lecture 31 implements this in full — and its job ends there: once the module returns to combinatorial benchmarks (Lecture 32 onward), solutions are built from discrete components again, and native ACO applies with no bridge needed at all. For now, the key idea is: **the archive replaces the pheromone as "what the colony remembers,"** and Gaussian sampling around archive members replaces component-by-component construction as "how a new candidate gets built."

1.5 Hand-Traced Example

Represent each solution as a 3-bit string $s = x_1x_2x_3$, built by deciding x_1, then x_2, then x_3. The components are the six assignments $x_j = v$ ($j \in \{1,2,3\}$, $v \in \{0,1\}$), and at step j the feasible set is $N(s^p) = \{x_j = 0,\ x_j = 1\}$. The objective values are:

s	000	001	010	011	100	101	110	111
$f(s)$	9	6	8	4	6	2	7	5

The global optimum is $s^* = 101$ ($f = 2$). The heuristic is a simple prior that 1s look good: $\eta_{x_j=1} = 2$ and $\eta_{x_j=0} = 1$ for every j — helpful for x_1 and x_3, but misleading for x_2, which is exactly the kind of imperfect rule of thumb pheromone is meant to correct. Use $\tau_o = 1$, $\alpha = 1$, $\beta = 1$, $\rho = 0.5$, $Q = 2$, and $m = 2$ ants.

Construction probabilities. With uniform pheromone, every step has the same probabilities, driven only by the heuristic:

$$p(x_j = 0) = \frac{1 \cdot 1}{1 \cdot 1 + 1 \cdot 2} = 0.333 \qquad p(x_j = 1) = \frac{1 \cdot 2}{1 \cdot 1 + 1 \cdot 2} = 0.667$$

Listing $x_j = 0$ first on the roulette wheel, an ant picks $x_j = 0$ if $\lambda < 0.333$ and $x_j = 1$ otherwise.

Each ant's construction, with pre-drawn λ at each step:

Ant	λ for x_1	λ for x_2	λ for x_3	s_k	$f(s_k)$	Deposit $Q/f(s_k)$
1	$0.60 \to 1$	$0.20 \to \mathbf{0}$	$0.90 \to 1$	101	2	$2/2 = 1.0$
2	$0.80 \to 1$	$0.50 \to 1$	$0.70 \to 1$	111	5	$2/5 = 0.4$

Pheromone update. Every component first evaporates, $\tau_c \leftarrow 0.5 \times 1 = 0.5$; then each ant deposits on the components of its own solution:

Component	$x_1 = 0$	$x_1 = 1$	$x_2 = 0$	$x_2 = 1$	$x_3 = 0$	$x_3 = 1$
Used by	—	ants 1, 2	ant 1	ant 2	—	ants 1, 2

Component	$x_1 = 0$	$x_1 = 1$	$x_2 = 0$	$x_2 = 1$	$x_3 = 0$	$x_3 = 1$
τ_c after update	0.5	$0.5 + 1.0 + 0.4 = 1.9$	$0.5 + 1.0 = 1.5$	$0.5 + 0.4 = 0.9$	0.5	$0.5 + 1.0 + 0.4 = 1.9$

Three things to notice: - Ant 1 chose $x_2 = 0$ **against** the heuristic's advice ($p = 0.333$) — a less likely but not impossible choice — and that is exactly what produced the global optimum $s^* = 101$. Probabilistic construction is ACO's exploration mechanism, the counterpart of SA accepting a worse move. - Components used by the *better* solution received the *larger* deposit: $x_2 = 0$ (used only by ant 1, $f = 2$) now carries more pheromone than $x_2 = 1$ (used only by ant 2, $f = 5$), even though the heuristic favoured $x_2 = 1$. - Components used by both ants ($x_1 = 1$, $x_3 = 1$) received two deposits, while components no ant used ($x_1 = 0$, $x_3 = 0$) only evaporated — with nothing to replace what faded.

1.6 In-Class Exercise

1.6.1 Exercise 1 — The Next Iteration

Using the pheromone after the update above, compute the construction probabilities for iteration 2, and trace one ant with draws $\lambda = (0.50,\ 0.40,\ 0.50)$. What would the same draws have produced in iteration 1?

Step	$\tau^\alpha \eta^\beta$ for $x_j = 0$	$\tau^\alpha \eta^\beta$ for $x_j = 1$	$p(x_j = 0)$	$p(x_j = 1)$	λ	Choice
x_1	$0.5 \times 1 = 0.5$	$1.9 \times 2 = 3.8$	$0.5/4.3 = 0.116$	0.884	0.50	1
x_2	$1.5 \times 1 = 1.5$	$0.9 \times 2 = 1.8$	$1.5/3.3 = 0.455$	0.545	0.40	**0**
x_3	$0.5 \times 1 = 0.5$	$1.9 \times 2 = 3.8$	$0.5/4.3 = 0.116$	0.884	0.50	1

This ant constructs $s = 101$ ($f = 2$), the global optimum. In iteration 1, the same draws would have produced 111 ($f = 5$): $\lambda = 0.40$ exceeds the original $p(x_2 = 0) = 0.333$, so the ant would have followed the heuristic to $x_2 = 1$. After a single update, pheromone has raised $p(x_2 = 0)$ from 0.333 to 0.455, so the colony is already starting to override a misleading heuristic. At the same time, pheromone has sharpened the choices the heuristic got right ($p(x_1 = 1)$ rose from 0.667 to 0.884). Both effects come from nothing but evaporation and quality-proportional deposit.

1.7 Take-Away Exercises

1.7.1 Exercise 1 — Switching Off Pheromone or Heuristic

Repeat Exercise 1's probability calculation with $\alpha = 0$ (ignore pheromone), and again with $\beta = 0$ (ignore the heuristic). With $\alpha = 0$, confirm that the iteration-2 probabilities are identical to iteration 1's, whatever the colony has learned. With $\beta = 0$, what are the iteration-1 probabilities,

before any ant has deposited anything? What does each extreme predict about how ACO would behave over many iterations on this problem?

1.7.2 Exercise 2 — Instantiating ACO for Routing

Consider a 4-stop routing instance — a depot (node 0) and three stops (1, 2, 3) — with distances $d_{01} = 2$, $d_{02} = 3$, $d_{03} = 4$, $d_{12} = 3$, $d_{13} = 5$, $d_{23} = 2$. Using the routing row of this lecture's instantiation table (components are edges (i, j), the feasible set is the edges to unvisited stops, $\eta_{ij} = 1/d_{ij}$), with $\tau_o = 1$, $\alpha = 1$, $\beta = 2$: 1. Compute the probabilities of each stop being chosen first from the depot, and use the draws $\lambda = (0.5,\ 0.5)$ for the first two choices to construct one complete tour that returns to the depot. What is its length? 2. Apply evaporation ($\rho = 0.5$) and this ant's deposit ($Q = 10$) to every edge. Which edges end up more attractive than they started, and which less? 3. Which parts of the generic pseudocode had to change to handle routing, and which stayed exactly the same?

1.8 Circling Back

- **Lecture 23 (Metaheuristics)**: Ant Colony Optimization is this module's example of the *swarm intelligence* paradigm — coordinated through a shared external memory (pheromone) rather than through any agent directly observing the others.
- **Lecture 24 (Simulated Annealing: Motivation & Pseudocode)**: SA's exploration comes from occasionally *accepting* a worse move; ACO's comes from occasionally *constructing* a less-preferred component, as ant 1 did with $x_2 = 0$ in this lecture's example.
- **Lecture 27 (Genetic Algorithm: Motivation & Pseudocode)**: both GA and ACO keep something that persists and builds up improvement across iterations — a population there, pheromone here — but GA's population *is* a set of candidate solutions, while ACO's pheromone is not a solution at all, only a bias on how future solutions get built. And just as GA's selection, crossover, mutation, and replacement operators are pluggable, so are ACO's components, feasible set, and heuristic.

1.9 Moving Forward

- **Lecture 31 (Ant Colony Optimization: Algorithm)**: implements ACOR — the continuous-domain bridge introduced above — in Python, and applies it to the same Ackley function Lectures 25 and 28 already used. ACOR's role ends there; Lecture 32 moves to real TSPLIB routing benchmarks, where ACO's native form — this lecture's pseudocode, instantiated with edges as components and $\eta_{ij} = 1/d_{ij}$ — applies directly.

1.10 Further Reading

- Dorigo, M. (1992). *Optimization, Learning and Natural Algorithms*. PhD thesis, Politecnico di Milano — the origin of Ant System, ACO's first instantiation.
- Dorigo, M., Maniezzo, V., and Colorni, A. (1996). "Ant System: Optimization by a Colony of Cooperating Agents." *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1), 29-41.

- Dorigo, M. and Di Caro, G. (1999). "The Ant Colony Optimization Meta-Heuristic." In *New Ideas in Optimization*, McGraw-Hill, 11-32 — the generic, problem-independent formulation of ACO (solution components and construction graphs) this lecture follows.
- Dorigo, M. and Stützle, T. (2004). *Ant Colony Optimization*. MIT Press — the standard reference text, including applications well beyond routing.
- Socha, K. and Dorigo, M. (2008). "Ant Colony Optimization for Continuous Domains." *European Journal of Operational Research*, 185(3), 1155-1173 — the ACOR extension previewed above, developed in Lecture 31.